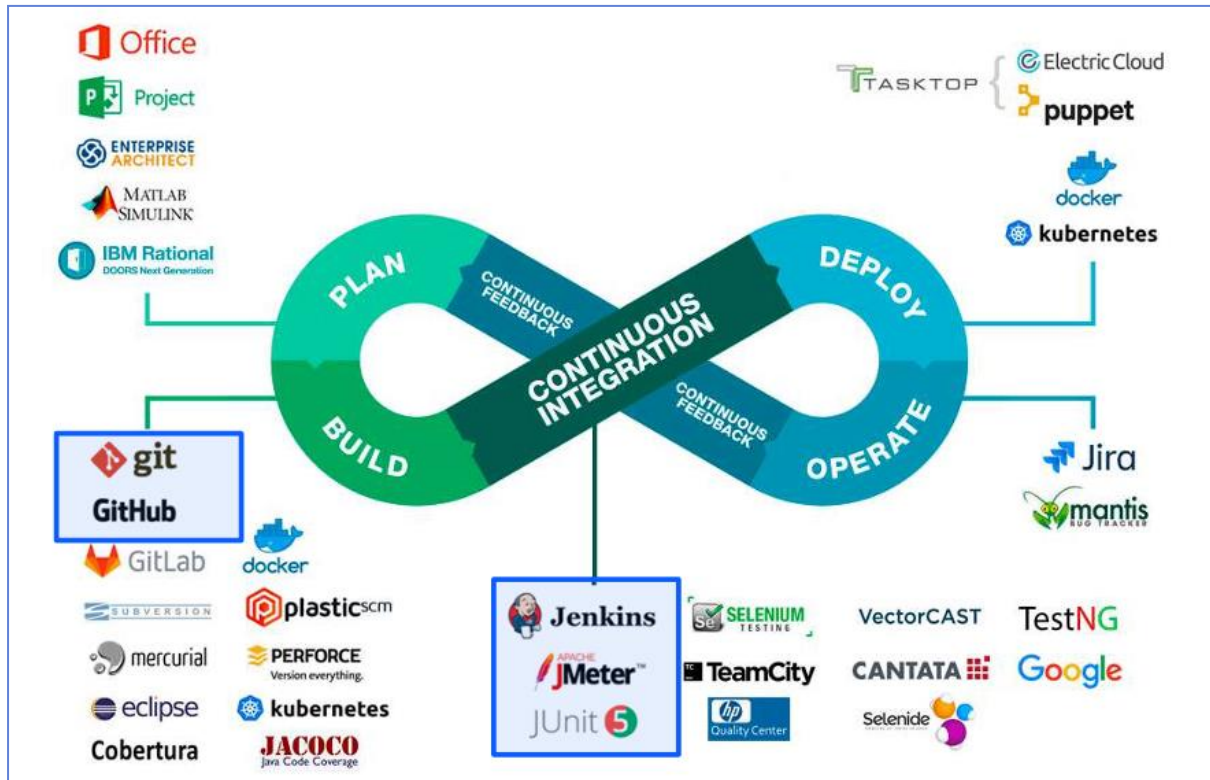




DevOps란?

DevOps는 소프트웨어 개발(Development)과 IT 운영(Operations) 간의 협업과 통합을 촉진하는 문화이자 방법론이다.

<https://www.netapp.com/ko/devops/what-is-devops/>

- ▶ DevOps는 애플리케이션 개발의 품질과 속도를 개선하고 **신규 또는 수정된 소프트웨어 기능이나 제품의 릴리즈 주기 단축을 장려하는 새로운 철학이자 프레임워크**입니다. DevOps 사례는 애플리케이션 개발 팀(Dev)과 해당 IT 운영 팀(Ops) 팀 간의 **원활하고 지속적인 커뮤니케이션, 협업, 통합, 가시성 및 투명성**을 장려합니다. "Dev"와 "Ops" 간의 이러한 긴밀한 관계는 초기 소프트웨어 계획부터 코딩, 구축, 테스트 및 릴리즈 단계와 구축, 운영 및 지속적인 모니터링에 이르는 DevOps 라이프사이클의 모든 단계에 걸쳐 계속됩니다. 이러한 관계는 추가 개선, 개발, 테스트 및 구축에 대한 지속적인 고객 피드백 루프를 추진하는 원동력이 됩니다. 이러한 노력이 제공하는 결과 중 하나는 필요한 기능 변경 또는 추가 기능을 더 빠르고 지속적으로 릴리즈할 수 있다는 것입니다.

DevOps 정의

<https://azure.microsoft.com/ko-kr/resources/cloud-computing-dictionary/what-is-devops/>

- ▶ 개발(Dev)과 운영(Ops)의 합성어인 DevOps는 **고객에게 지속적으로 가치를 제공하도록 지원하는 사람, 프로세스 및 기술의 합집합**입니다. 팀에는 DevOps가 어떤 의미일까요? DevOps는 개발, IT 운영, 품질 엔지니어링, 보안 등 이제껏 서로 단절되었던 역할들이 서로 조율하고 협업하여 더욱 안정적이고 뛰어난 제품을 생산할 수 있도록 지원합니다. DevOps 방식과 도구를 비롯한 DevOps 문화를 도입하는 팀은 고객 요구 사항에 보다 효과적으로 대응하고, 더욱 안심하고 애플리케이션을 빌드하며, 비즈니스 목표를 더 빨리 달성할 수 있습니다.

DevOps 이해

<https://www.redhat.com/ko/topics/devops>

- ▶ DevOps는 신속한 고품질 서비스 제공을 통해 **비즈니스 가치와 대응력을 향상시키기 위한 기업 문화, 자동화, 플랫폼 설계에 대한 접근 방식**입니다. 이러한 접근 방식은 빠르게 반복적으로 IT 서비스를 제공하는 역량이 있어야 가능합니다. DevOps는 레거시 애플리케이션과 최신 클라우드 네이티브 애플리케이션 및 인프라를 연결하는 것을 의미합니다.

결론적으로

DevOps는 **2000년대부터** 나온 용어로 보통 **문화**라고 표현한다.

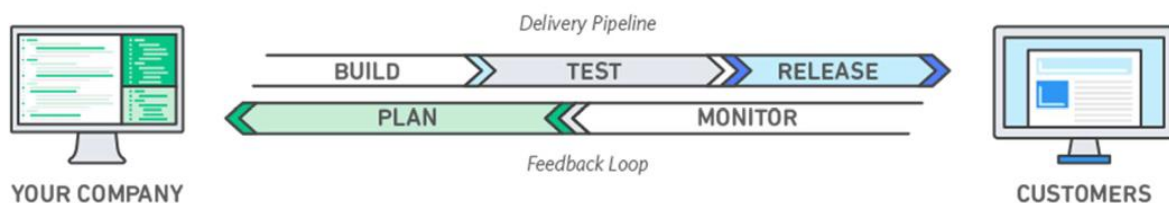
고객의 입장에서 빠르게 니즈를 해결 해주면 신뢰도가 높아 질것이므로 **사람의 손을 안타고 빠르게 고객에게 응대 해주자는 컨셉이 DevOps 의 한 축이다.**

즉, DevOps는 기업의 비즈니스를 지속, 가속 및 성장 시키기 위한 다양한 디지털 전략의 집합이다.

DevOps 모델

DevOps 모델 정의

- ▶ DevOps는 **애플리케이션과 서비스를 빠른 속도로 제공할 수 있도록 조직의 역량을 향상시키는 문화 철학, 방식 및 도구의 조합**입니다. 기존의 소프트웨어 개발 및 인프라 관리 프로세스를 사용하는 조직보다 제품을 더 빠르게 혁신하고 개선할 수 있습니다. 이러한 빠른 속도를 통해 조직은 고객을 더 잘 지원하고 시장에서 좀 더 효과적으로 경쟁할 수 있습니다.



DevOps 작동 방식

- ▶ DevOps 모델에서는 **개발팀과 운영팀이 더 이상 "사일로"에 묶여 있지 않습니다.** 때로는 이 두 팀이 단일팀으로 병합되어 엔지니어가 개발에서 테스트, 배포, 운영에 이르기까지 전체 애플리케이션 수명 주기에 걸쳐 작업하고 단일 기능에 한정되지 않은 광범위한 기술을 개발합니다.
- ▶ 일부 DevOps 모델에서 품질 보증팀과 보안팀 또한 애플리케이션 수명 주기에 걸쳐 개발 및 운영과 좀 더 긴밀하게 통합됩니다. DevOps 팀 전체가 보안을 중점으로 두는 경우 때때로 DevSecOps라고 불립니다.
- ▶ 이러한 팀에서는 DevOps 방식을 사용하여 속도가 느리고 수동으로 수행되던 프로세스를 자동화합니다. 또한, 애플리케이션을 안정적으로 빠르게 운영하고 개선하는 데 도움이 되는 기술 스택과 도구를 사용합니다. 이러한 도구 덕분에 엔지니어는 이전 같았으면 다른 팀의 도움이 필요했을 코드 배포 또는 인프라 프로비저닝과 같이 작업을 독립적으로 수행할 수 있으며, 따라서 팀의 작업 속도가 더욱 빨라집니다.

DevOps 모범사례

	지속적 통합 CI	지속적 통합은 자동화된 빌드 및 테스트가 수행된 후, 개발자가 코드 변경 사항을 중앙 리포지토리에 정기적으로 병합하는 소프트웨어 개발 방식입니다. 지속적 통합의 핵심 목표는 버그를 신속하게 찾아 해결하고, 소프트웨어 품질을 개선하고, 새로운 소프트웨어 업데이트를 검증 및 릴리스하는 데 걸리는 시간을 단축하는 것입니다.
	지속적 전달 CD	지속적 전달은 프로덕션에 릴리스하기 위한 코드 변경이 자동으로 빌드, 테스트 및 준비되는 소프트웨어 개발 방식입니다. 빌드 단계 이후의 모든 코드 변경 사항을 테스트 환경 및/또는 프로덕션 환경에 배포함으로써 지속적 통합을 확장합니다. 지속적 전달이 적절하게 구현되면, 개발자는 언제나 즉시 배포할 수 있고 표준화된 테스트 프로세스를 통과한 빌드 아티팩트를 보유하게 됩니다.
	마이크로 서비스	마이크로 서비스 아키텍처는 단일 애플리케이션을 작은 서비스의 집합으로 구축하는 설계 접근 방식입니다. 각 서비스는 자체 프로세스에서 실행되고, 주로 HTTP 기반 API(애플리케이션 프로그래밍 인터페이스)라는 간편한 매커니즘을 사용하는 잘 정의된 인터페이스를 통해 다른 서비스와 통신합니다. 마이크로 서비스는 비즈니스 기능을 중심으로 구축되며, 각 서비스는 단일 목적으로 한정되어 있습니다. 다양한 프레임워크 또는 프로그래밍 언어를 사용하여 마이크로 서비스를 작성하고, 이를 독립적으로 단일 서비스 또는 서비스 그룹으로 배포할 수 있습니다.
	코드형 인프라	코드형 인프라는 버전 관리 및 지속적 통합과 같은 코드와 소프트웨어 개발 기술을 사용하여 인프라를 프로비저닝하고 관리하는 방식입니다. 클라우드의 API 중심 모델을 사용하면 개발자와 시스템 관리자가 수동으로 리소스를 설정 및 구성할 필요 없이 프로그래밍 방식으로 대규모로 인프라와 상호 작용할 수 있습니다. 따라서 엔지니어는 코드 기반 도구를 사용하여 인프라와 인터페이스하고, 애플리케이션 코드를 다루는 방법과 유사한 방식으로 인프라를 다룰 수 있습니다. 인프라가 코드를 통해 정의되므로 인프라와 서버를 표준화된 패턴을 사용하여 배포하고, 최신 패치와 버전으로 업데이트하거나, 반복 가능한 방식으로 복제할 수 있습니다.
	모니터링 및 로깅	조직은 지표와 로그를 모니터링하여 애플리케이션 및 인프라 성능이 제품의 최종 사용자 경험에 어떤 영향을 미치는지 확인합니다. 조직은 애플리케이션과 인프라에서 생성되는 데이터 및 로그를 캡처하고 분류한 다음 이를 분석함으로써 변경 또는 업데이트가 사용자에게 어떤 영향을 주는 지 이해하고, 문제의 근본 원인 또는 예상치 못한 변경에 대한 통찰력을 확보합니다. 서비스는 연중무휴 24시간 사용할 수 있어야 하고 애플리케이션 및 인프라 업데이트 빈도가 증가함에 따라 적극적인 모니터링이 점점 더 중요해지고 있습니다. 이러한 데이터에 대한 실시간 분석을 수행하거나 알림을 생성하는 것도 조직이 좀 더 능동적으로 서비스를 모니터링하는 데 도움이 됩니다.
	커뮤니케이션 및 협업	조직에서 커뮤니케이션과 협업이 증가하는 것도 DevOps의 주요 문화적 측면 중 하나입니다. DevOps 도구 및 소프트웨어 제공 프로세스 자동화를 사용하면 개발 및 운영의 워크플로와 책임을 물리적으로 합침으로써 협업이 이루어집니다. 해당 팀에서는 이 위에 채팅 애플리케이션, 문제 또는 프로젝트 추적 시스템, wiki를 사용하여 커뮤니케이션을 지원하고 정보를 공유하는 강력한 문화적 표준을 확립합니다. 이를 통해 개발자와 운영 그리고 마케팅이나 영업과 같은 다른 팀 간에도 커뮤니케이션이 활발해지면서 조직의 모든 부분에서 목표와 프로젝트에 좀 더 가깝게 다가갈 수 있습니다.

Devops 문화를 정착하는데 대표적인 Best Practice(모범사례) 는

1. 지속적통합, 지속적전달
2. 코드형 인프라 – 테라폼

Devops 엔지니어를 위한 CI/CD Pipeline

■ Continuous Integration

▶ 지속적 통합

- 지속적 통합은 자동화된 빌드 및 테스트가 수행된 후, 개발자가 코드 변경 사항을 중앙 리포지토리에 정기적으로 병합하는 데브옵스 소프트웨어 개발 방식입니다. 지속적 통합은 소프트웨어 릴리스 프로세스 중 빌드 또는 통합 단계를 주로 가리키며, 자동화 구성 요소(예: CI 또는 빌드 서비스)와 문화적 구성 요소(예: 빈번하게 통합하도록 학습) 모두를 포함합니다. 지속적 통합의 핵심 목표는 버그를 신속하게 찾아 해결하고, 소프트웨어 품질을 개선하고, 새로운 소프트웨어 업데이트를 검증 및 릴리스하는 데 걸리는 시간을 단축하는 것입니다.

▶ 왜 지속적 통합이 필요합니까?

- 이전에는 팀의 개발자가 장기간 격리된 상태로 작업하고, 작업이 완료된 후에야 변경 사항을 마스터 브랜치에 병합했습니다. 이로 인해 병합 코드가 어렵고 시간 소모적으로 변하게 되었고 또한, 수정 없이 오랫동안 버그가 축적되는 결과가 나타났습니다. 이러한 요소 때문에 고객에게 업데이트를 신속하게 제공하기가 더 어려워졌습니다.

▶ 지속적 통합은 어떻게 작동합니까?

- 지속적 통합에서는 개발자가 Git과 같은 버전 관리 제어 시스템을 사용하여 공유된 리포지토리에 빈번하게 커밋하게 됩니다. 각 커밋에 앞서, 개발자는 통합 전에 추가 검증 계층으로써 코드에 로컬 유닛 테스트를 수행할 수 있습니다. 지속적 통합 서비스는 새로운 코드 변화에 대한 유닛 테스트를 자동으로 구축하고 실행하여 즉시 모든 오류를 표면화합니다.

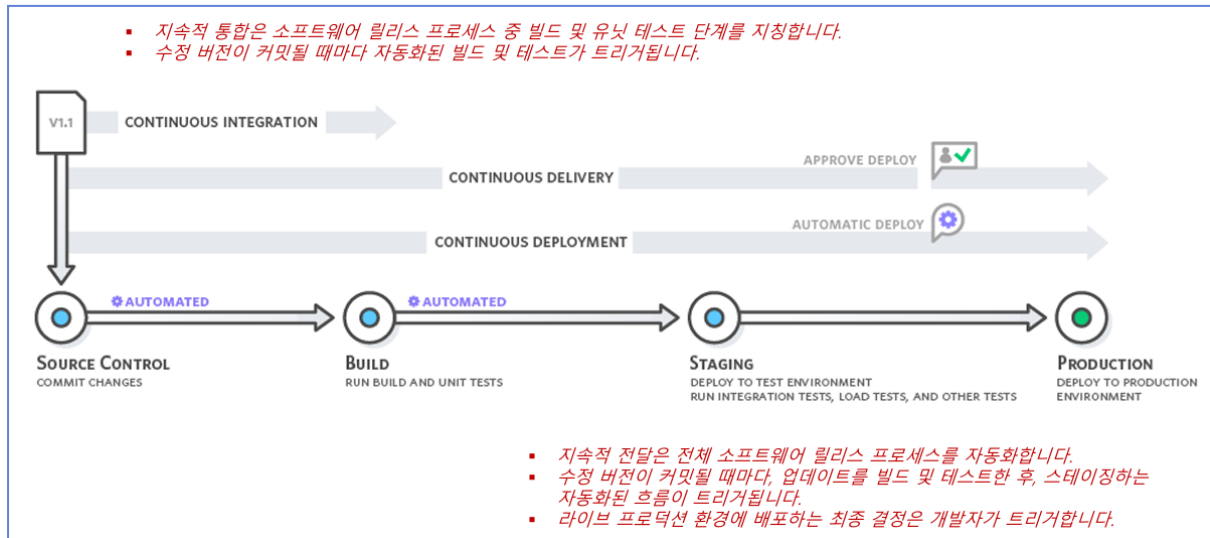
■ Continuous Delivery (Deployment)

▶ 지속적 전달

- 지속적 전달(Continuous Delivery)은 프로덕션에 릴리스하기 위한 코드 변경이 자동으로 준비되는 소프트웨어 개발 방식입니다. 현대 애플리케이션 개발의 기반인 지속적 전달은 빌드 단계 이후의 모든 코드 변경을 테스트 환경 및/또는 프로덕션 환경에 배포함으로써 지속적 통합을 확장합니다. 적절하게 구현할 경우, 개발자는 언제나 즉시 배포할 수 있고 표준화된 테스트 프로세스를 통과한 빌드 아티팩트를 보유할 수 있습니다.
- 지속적 전달에서는 개발자가 단순한 유닛 테스트 외에도 다양한 테스트를 자동화할 수 있으므로, 고객에게 배포하기 전에 여러 차원에서 애플리케이션 업데이트를 확인할 수 있습니다. 이러한 테스트에는 UI 테스트, 로드 테스트, 통합 테스트, API 안정성 테스트 등이 포함될 수 있습니다. 이를 통해 개발자는 업데이트를 좀 더 철저히 검증하고 문제를 사전에 발견할 수 있습니다. 온프레미스에서는 힘들었지만, 클라우드에서는 테스트용으로 여러 개의 환경을 생성하고 복제하는 작업을 비용 효율적으로 손쉽게 자동화할 수 있습니다.

▶ 지속적 전달(Continuous Delivery)과 지속적인 개발

- 지속적 전달에서는 모든 코드 변경이 빌드 및 테스트된 후, 비프로덕션 테스트 또는 스테이징 환경으로 푸시됩니다. 프로덕션 배포 전에 여러 개의 병렬 테스트 단계가 있을 수 있습니다. 지속적 전달과 지속적 배포의 차이점은 프로덕션에 업데이트에 대한 수동 승인 존재 여부입니다. 지속적 배포의 경우 명시적 승인 없이 자동으로 프로덕션이 일어납니다.



CI/CD 파이프라인 (현실세계에서는 파이프라인은 수도배관이다)

CI(Continuous Integration) - 사람의 손을 안타고 자동으로 빌드하고 테스트는 하는 것을 말한다.

Continuous deployment - statging(배포)

Continuous delivery - 사람의 손이 들어간다. 특히나 금융권 서버는 사람의 승인이 필요하다.

이 부분은 좀 Critical하기 때문에 운영서버까지 배포하는 것은
마지막에 반드시 사람의 승인을 걸쳐서 배포하는 경우이다.

DevOps 프로세스와 툴

<https://www.opsera.io/blog/top-25-devops-tools-that-you-need-to-know>

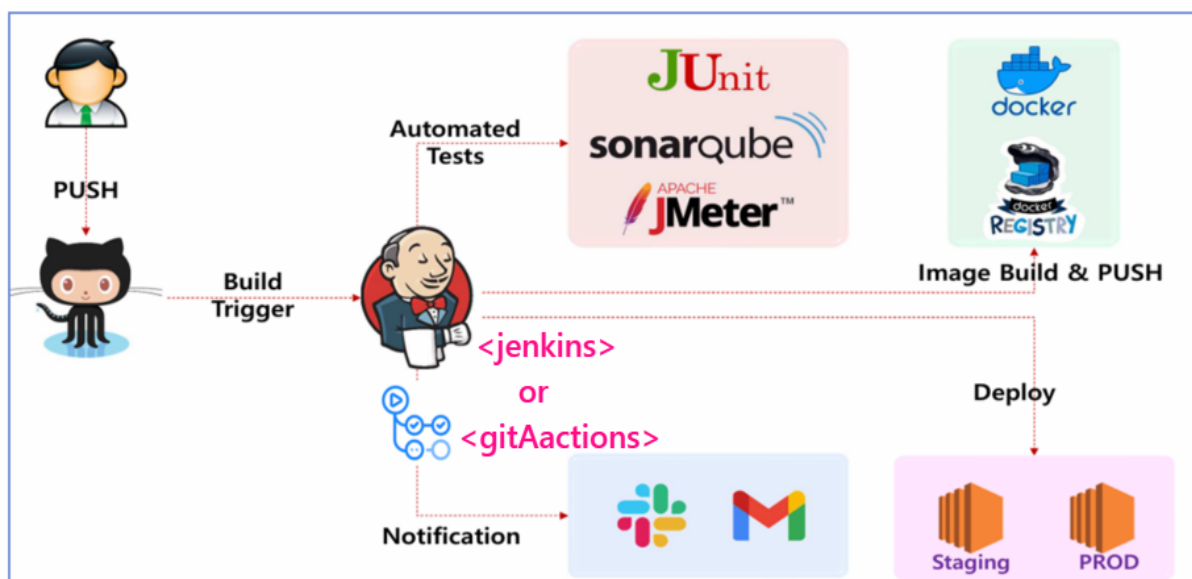
무한 반복과 수많은 툴들의 집합



CI/CD를 하는 사람들은 즉 파이프라인으로 구축하는 사람들이 저 많은 툴을 알아야하는가?

그러기 힘들다. CI/CD를 하는 사람들은 각 영역의 담당자와 대화 정도 할 수준이면 된다.

CI/CD파이프라인(프로세스,워크플로우) – 우리가 할 일



CI/CD란?

Continuous Integration, Continuous Deployment

서비스를 운영 과정에서 여러 기능을 수정하고 추가한 후에 배포를 하게 되는데 이 때 **수정한 코드에 대한 테스트, 통합 과정, 그리고 배포의 과정을 자동화 하는 것을 의미한다.**

또한 소스의 개발과 배포과정의 갭, 즉 통합시간을 최소화하자는 것도 포함한다.

이 반복적인 작업을 자주 해야 하기 때문에 배포작업을 자동화하려는 것이다.

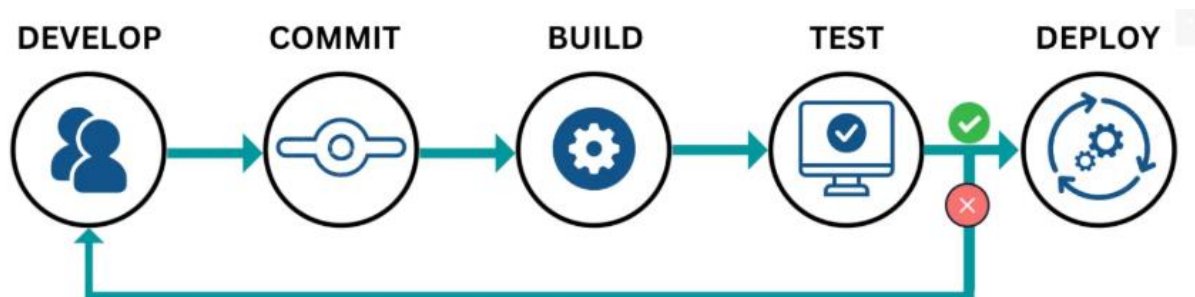
쉽게 표현하자면 CI/CD 는 테스트(Test), 통합(Merge), 배포(Deploy)의 과정을 자동화하는 걸 의미한다.

CI/CD 를 왜 배우는 걸까?

서비스를 운영하다보면 새로운 기능을 추가하는 일이 많아진다. 새로운 기능에 대한 코드를 작성한 뒤에 Commit 을 한다. 그런 뒤에 브랜치에 Merge 를 하고 배포를 한다. 배포를 할 때 직접 컴퓨터 서버(ex. AWS EC2)에 접속해서 새로운 코드를 다운받아 실행시켜주어야 한다.

이 과정을 코드의 수정이 일어날 때마다 반복하기란 너무 귀찮은 일이다. 그래서 **이런 반복적인 과정을 자동화시키기 위해 CI/CD 를 배우는 것이다.**

CI/CD 과정은 일반적으로 다음과 같은 과정으로 일어난다.



출처 : <https://www.performetriks.com/post/load-test-in-a-cicd-environment>

특정 기능을 개발 완료해서 Commit을 찍으면 빌드가 되게 셋팅한다. 빌드가 완료되면 작성한 테스트 코드를 실행시킨다.(테스트 코드를 작성하지 않은 서비스에서는 이 과정을 생략하기도 한다.) 그런 뒤 테스트가 통과하면 실제 서버 컴퓨터에 최신 코드가 배포된다.

CI/CD 자동화 도구

- ✓ Github Actions
- ✓ Jenkins
- ✓ Circle CI

✓ Travis CI

현업에서 Github Actions 뿐만 아니라 Jenkins도 많이 활용한다. Github Actions과 Jenkins 둘 중에 하나만 쓰더라도 필요한 CI/CD 구성을 전부 할 수 있다. Github Actions를 사용할 지, Jenkins를 사용할 지는 장단점을 비교해서 상황에 맞게 선택하면 된다. Github Actions 만을 이용해도 충분히 CI/CD를 할 수 있다.

Jenkins는 별도의 서버 구축을 해야 한다는 단점이 있다. 이 때문에 서버를 빌리는 비용이 발생하게 된다.

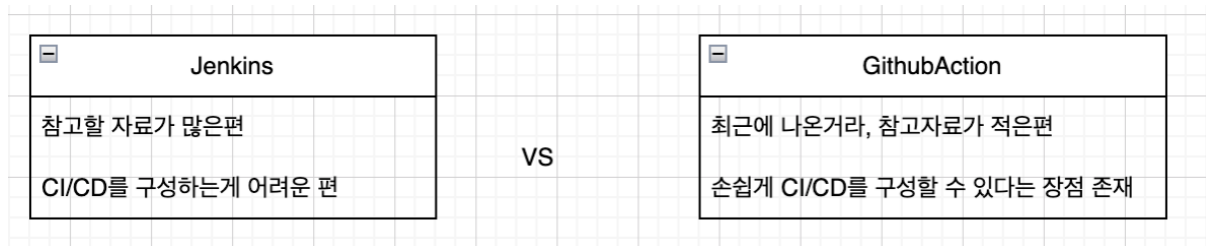
하지만

Github Actions는 별도의 서버 구축 없이 Github에 내장되어 있는 Github Actions 기능을 사용할 수 있다. 비용적인 측면도 유리하고 셋팅하는 데 시간을 쓸 필요도 없다.

아직 많은 현업에서 jenkins를 사용하고 있지만 점점 Github Actions도 많이 사용하고 있다.

카카오엔터프라이즈가 GitHub Actions를 사용하는 이유

<https://tech.kakao.com/posts/516>

**Github Actions를 활용한 전체적인 CI/CD 흐름**

Github Actions를 “**로직을 실행시킬 수 있는 일종의 컴퓨터**”라고 생각하면 된다. CI/CD 과정에서 Github Actions는 “빌드, 테스트, 배포”에 대한 로직을 실행시키는 역할을 하게 된다.



1. 코드 작성 후 **Commit**
2. Github 에 **Push**
3. Push 를 감지해서 Github Actions 에 작성한 로직이 실행
 1. 빌드(**Build**)
 2. 테스트(**Test**)

3. 서버로 배포(Deploy)
4. 서버에서 배포된 최신 코드로 서버를 재실행

Github Actions 기본 문법

Github Actions 설명서

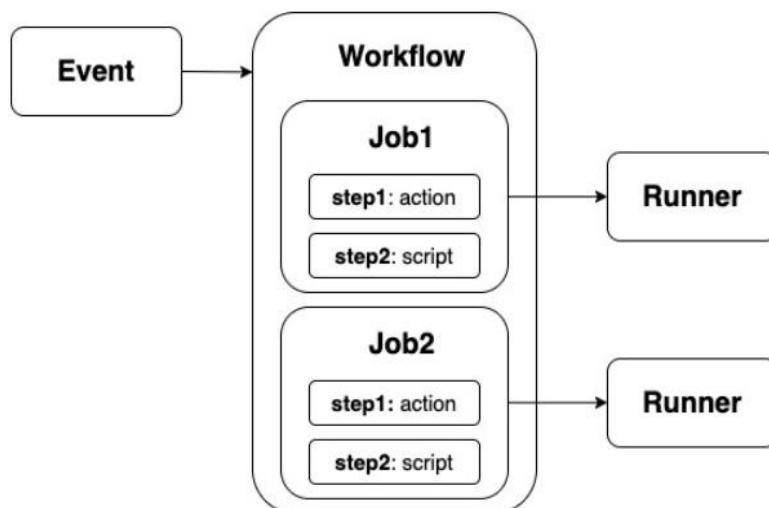
공식문서 : <https://docs.github.com/ko/actions>

Github Actions를 실행시키기 위해서는 반드시 `.github/workflows`라는 디렉터리에 `.yaml` 또는 `.yml`의 확장자로 파일을 작성해야 한다. 그리고 `.github/workflows`는 프로젝트 폴더의 최상단에 만들어야 한다.

하나의 workflow는 한 개 이상의 job으로 구성된다.

하나의 job은 한 개 이상의 step(특정작업을 수행하는 가장 작은 단위)으로 구성된다.

참고 : <https://jh-labs.tistory.com/416>



Github Actions를 이용한 CI/CD workflow 실습해보기

시나리오

- Github에 cicd_01 프로젝트 만들기
- Actions 클릭 > workflow 작성하기
- 확인하기

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
[Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner *

8253jang

Repository name *

cicd_01

✓ cicd_01 is available.

8253jang / cicd_01

클릭

Code Issues Pull requests **Actions** Projects Wiki Security Insights Settings

Please configure another 2FA method to reduce your risk of permanent account lockout. If you use SMS for 2FA, we strongly recommend against SMS as it is prone to fraud and delivery may be

Get started with GitHub Actions

Build, test, and deploy your code. Make code reviews, branch management, and issue triaging work the way you want. Select a workflow to get started.

Skip this and [set up a workflow yourself](#) →

클릭

cicd_01 / .github / workflows / main.yml in main

Cancel changes

Commit changes...

클릭

```
1 # Workflow의 이름
2 # Workflow : 하나의 yaml 파일을 하나의 Workflow라고 부른다.
3 name: Github Actions 실행시켜보기
4
5 # Event : 실행되는 시점을 설정
6 # main이라는 브랜치에 push 될 때 아래 Workflow를 실행
7 on:
8   push:
9     branches:
10      - main
11
12 # 하나의 workflow는 1개 이상의 job으로 구성된다.
13 # 여러 job은 기본적으로 순차적으로 실행된다.
14 jobs:
15   # job을 실행하기 위한 id
16   my-deploy-job:
17     # github actions를 실행시킬 서버 종류 선택
18     runs-on: ubuntu-latest
19
20 # Step : 특정 작업을 수행하는 가장 작은 단위
21 # job은 여러 Step들로 구성되어 있다.
22 steps:
23   - name: Hello World 띄우기 # Step이 이름 붙여지는 기능
24     run: echo "Hello World" # 실행시킬 명령어 작성
25
26   - name: 여러 명령어 분장 작성하기
27     run: |
28       echo "Have a nice day"
29       echo "Good luck"
```

→ 제공한 ~.yml정보 복사해서
넣기

main.yml

```
# Workflow의 이름
# Workflow : 하나의 yaml 파일을 하나의 Workflow라고 부른다.
name: Github Actions 실행시켜보기

# Event : 실행되는 시점을 설정
# main이라는 브랜치에 push 될 때 아래 Workflow를 실행
on:
  push:
    branches:
```

- main

하나의 Workflow는 1개 이상의 Job으로 구성된다.

여러 Job은 기본적으로 병렬적으로 수행된다.

jobs:

Job을 식별하기 위한 id

My-Deploy-Job:

Github Actions를 실행시킬 서버 종류 선택

runs-on: ubuntu-latest

Step : 특정 작업을 수행하는 가장 작은 단위

Job은 여러 Step들로 구성되어 있다.

steps:

- name: Hello World 찍기 # Step에 이름 붙이는 기능
run: echo "Hello World" # 실행시킬 명령어 작성

- name: 여러 명령어 문장 작성하기

run: |
echo "Have a nice day"
echo "Good luck"

The top screenshot shows the GitHub Actions interface for the repository '8253jang / cicd_01'. The 'Actions' tab is selected, and the 'All workflows' page is displayed. A workflow run 'Create main.yml' is highlighted, showing it was triggered by a push to the 'main' branch and is currently 'Queued'.

The bottom screenshot shows the 'Summary' page for the 'Create main.yml' workflow. The 'Jobs' section is expanded, and the 'My-Deploy-Job' is highlighted. The job status is 'Success' and it took 3s to complete.

My-Deploy-Job
succeeded 1 minute ago in 3s

- > ☒ Set up job
- > ☒ Hello World 찍기
- > ☒ 여러 명령어 문장 작성하기
- > ☒ Complete job

My-Deploy-Job
succeeded 4 minutes ago in 3s

☒ Set up job

```

1 Current runner version: '2.323.0'
2 ▶ Operating System
6 ▶ Runner Image
11 ▶ Runner Image Provisioner
13 ▶ GITHUB_TOKEN Permissions
17 Secret source: Actions
18 Prepare workflow directory
19 Prepare all required actions
20 Complete job name: My-Deploy-Job → job을 식별하기 위한 id

```

☒ Hello World 찍기 → name

```

1 ▶ Run echo "Hello World"
4 Hello World

```

☒ 여러 명령어 문장 작성하기 → name

```

1 ▶ Run echo "Have a nice day"
5 Have a nice day
6 Good luck

```

☒ Complete job

```

1 Cleaning up orphan processes

```

Gihub Acations의 변수 활용

.gitignore 로 application.yml or application.properties 설정파일이나 보안적으로 중요한 값은 것에 올리지 않는다.

하지만 build 할 때는 그 정보가 필요하다. 이럴 때 어떻게 해야 할까?

1. Github Actions 자체에 저장되어 있는 변수를 사용해보자.

2. Actions 의 Secret 변수 활용

Secret 변수 – 설정 값 등 노출되면 안되는 값을 저장할 때 사용

한 번 저장하면 수정화면에서도 보이지 않고 출력화면에서도 보이지 않음.

.gitignore 로 github에 올리지 않은 시스템 id와 password 같은 정보를

저장하는 용도로 많이 사용함

저장 한 후에 조회하는 방법은 없다.

: Settings > Secrets and Variables 로 이동 > Actions 클릭 > Secrets tab의 New repository secret 클릭

변수활용 방법 2가지 실습해보기

Github Actions 자체에 저장되어 있는 변수

위에서 실습한 cicd-01 프로젝트의 .github/workflows/main.yml 수정한다.

제공소스 > main_02.yml

#참고: <https://docs.github.com/en/actions/learn-github-actions/variables>

- name: Github Actions 자체에 저장되어 있는 변수 사용해보기

run: |

echo \$GITHUB_SHA

echo \$GITHUB_REPOSITORY

```

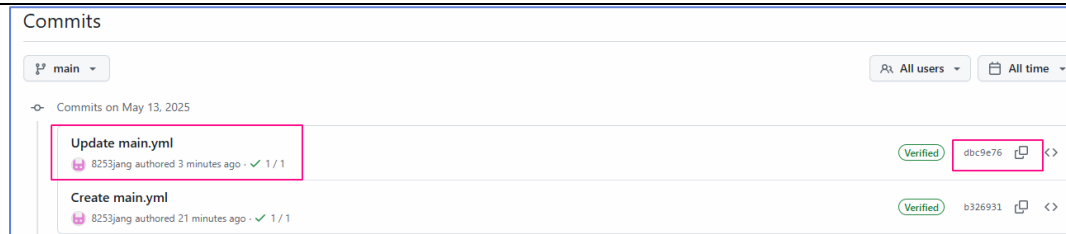
1  # Workflow의 이름
2  # Workflow : 하나의 yaml 파일을 하나의 Workflow라고 부른다.
3  name: Github Actions 실행시켜보기
4
5  # Event : 실행되는 시점을 설정
6  # main이라는 브랜치에 push 할 때 아래 Workflow를 실행
7  on:
8    push:
9      branches:
10       - main
11
12 # 하나의 workflow는 1개 이상의 Job으로 구성된다.
13 # 여러 Job은 기본적으로 병렬적으로 수행된다.
14 jobs:
15   # Job을 식별하기 위한 id
16   My-Deploy-Job:
17     # Github Actions를 실행시킬 서버 종류 선택
18     runs-on: ubuntu-latest
19
20     # Step : 특정 작업을 수행하는 가장 작은 단위
21     # Job은 여러 step들로 구성되어 있다.
22     steps:
23       - name: Hello World 찍기 # Step에 이름 붙이는 기능
24         run: echo "Hello World" # 실행시킬 명령어 작성
25
26       - name: 여러 명령어 문장 작성하기
27         run: |
28           echo "Have a nice day"
29           echo "Good luck"
30
31       # 참고: https://docs.github.com/en/actions/learn-github-actions/variables
32       - name: Github Actions 자체에 저장되어 있는 변수 사용해보기
33         run: |
34           echo $GITHUB_SHA
35           echo $GITHUB_REPOSITORY
36

```

위 빨간 부분을 추가하고 오른쪽 상단의 녹색 Commit changes를 클릭한다.
Actions 클릭한다.

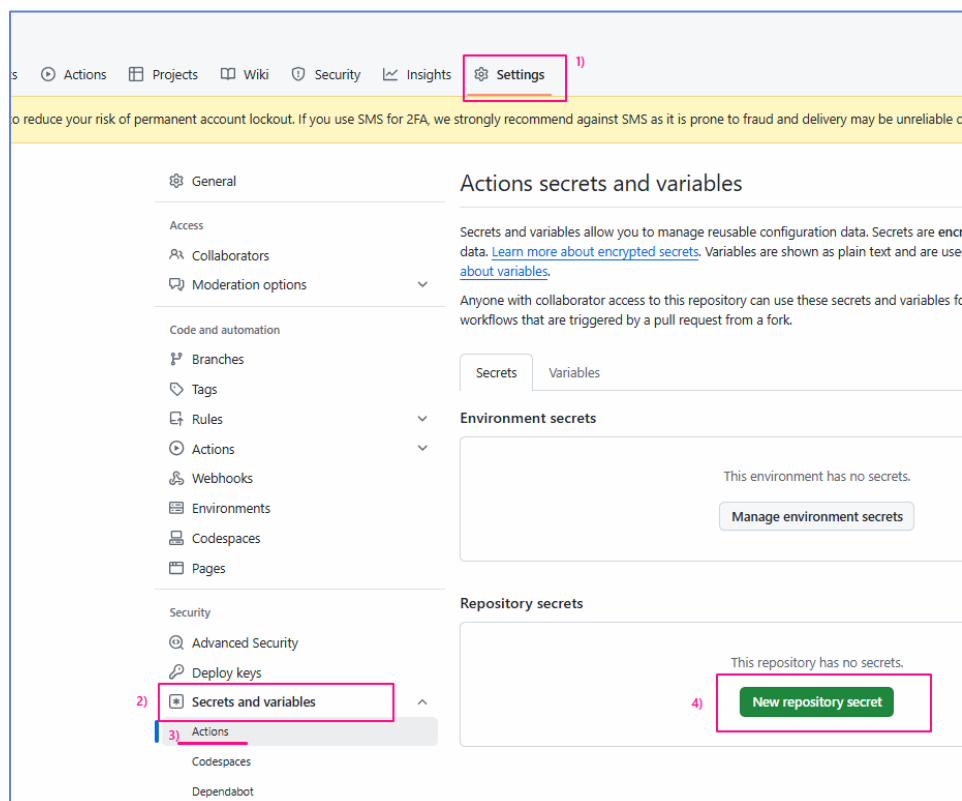
My-Deploy-Job
succeeded 2 minutes ago in 4s

- > ✓ Set up job
- > ✓ Hello World 찍기
- > ✓ 여러 명령어 문장 작성하기
- ✓ Github Actions 자체에 저장되어 있는 변수 사용해보기
 - 1 ▶ Run echo \$GITHUB_SHA
 - 5 dbc9e76e14a0e68054e3d7312b7a0c4782716f99 → \$GITHUB_SHA 의 값으로 커밋된 hash정보
 - 6 8253jang/cicd_01 → \$GITHUB_REPOSITORY 값으로 현재 저장소이름
- > ✓ Complete job



Actions 의 Secret 변수 활용

Settings > Secrets and Variables 로 이동 > Actions 클릭 > Secrets tab의 New repository secret 클릭



Actions secrets / New secret

Name *
MY_ENV_NAME

Secret *
KOSTA

Add secret

Actions secrets / New secret

Name *
MY_ENV_PASS

Secret *
1234

Add secret

Repository secrets

New repository secret

Name ↕	Last updated
MY_ENV_NAME	now
MY_ENV_PASS	now

main.yml에 적용하기 (main_03.yml참고)

```

31 # 참고: https://docs.github.com/en/actions/learn-github-actions/variables
32 - name: Github Actions 자체에 저장되어 있는 변수 사용해보기
33   run: |
34     echo $GITHUB_SHA
35     echo $GITHUB_REPOSITORY
36
37 - name: Github Actions Secret 변수 사용해보기
38   run: |
39     echo ${ secrets.MY_ENV_NAME }
40     echo ${ secrets.MY_ENV_PASS }
41
42
43

```

secret변수 사용할때 `${ secrets.이름 }`
사용자정의 secret변수 사용할때는 `${ 중괄호 2개 넣는다 }`

위 빨간 부분을 추가하고 오른쪽 상단의 녹색 Commit changes를 클릭한다.
Actions 클릭한다.

